

Trabalho Final de Complexidade de Algoritmos

Uso de Algoritmo Heurístico A* Para Encontrar Caminho Mais Curto Entre Quaisquer Dois Aeroportos

Alunos:
Matheus Azevedo de Sá
Rian Vulcanai

Professor:
LEANDRO ISRAEL PINTO

1 de dezembro de 2025

Conteúdo

1	Introdução	2
2	Definição do Problema	2
3	Solução Proposta	2
4	Apresentação e Pré-Processamento dos Dados	3
4.1	Bancos de Dados Originais	3
4.2	Fluxo de Funcionamento do <code>conversor.py</code>	3
5	Heurística e a Fórmula de Haversine	3
5.1	A Fórmula de Haversine	3
5.2	Justificativa da Heurística	4
6	Fluxo e Análise do <code>astar.cpp</code>	4
6.1	Fluxo de Funcionamento	4
6.2	Análise da Complexidade e Estruturas	5
7	Manual de Execução	5
8	Conclusão	5
9	Referências	6

1 Introdução

Este relatório apresenta a implementação e análise de um sistema para encontrar o caminho mais curto entre quaisquer dois aeroportos em uma rede de rotas globais. O projeto está dividido em duas etapas principais: a preparação dos dados geográficos, realizada pelo script Python `converter.py`, e a execução do algoritmo de busca, implementada em C++ (`astar.cpp`). O objetivo central é aplicar o **Algoritmo de Busca A*** (A^*), uma extensão do Algoritmo de Dijkstra que utiliza uma função **heurística** para otimizar a exploração do grafo, garantindo a otimalidade da solução com maior eficiência computacional.

2 Definição do Problema

O problema a ser resolvido é o **Problema do Caminho Mais Curto (Shortest Path Problem)** em um grafo ponderado e direcionado.

- **Vértices (V):** Representados pelos aeroportos, identificados pelo seu código IATA (ex.: GRU, LAX).
- **Arestas (E):** Representam rotas de voo diretas entre dois aeroportos. As arestas são direcionadas, pois uma rota $A \rightarrow B$ não implica necessariamente na existência da rota $B \rightarrow A$ (embora geralmente exista).
- **Pesos (Ponderação):** O peso de cada aresta é a **distância real em quilômetros (km)** entre o aeroporto de origem e o de destino.

O desafio reside em encontrar a sequência de voos que minimiza a soma total das distâncias percorridas (ou seja, o caminho de menor custo) entre um aeroporto inicial e um aeroporto final, em uma rede que pode ter milhares de nós e rotas.

3 Solução Proposta

A solução proposta utiliza o **Algoritmo de Busca A*** (A^*). O A^* combina os pontos fortes do Algoritmo de Dijkstra (que garante a otimalidade) com uma heurística que estima o custo restante até o destino, guiando a busca de forma eficiente.

O A^* minimiza o custo total estimado $f(n)$ para qualquer nó n , definido por:

$$f(n) = g(n) + h(n)$$

- $g(n)$: É o custo do caminho já percorrido do nó inicial até o nó atual n (a soma dos pesos das arestas, em km).
- $h(n)$: É a função heurística, a estimativa do custo do nó atual n até o nó objetivo.

Neste projeto, a heurística $h(n)$ é calculada pela **distância geodésica** (menor caminho entre dois pontos em uma esfera) entre o aeroporto atual e o destino, usando a **Fórmula de Haversine**.

4 Apresentação e Pré-Processamento dos Dados

4.1 Bancos de Dados Originais

Os dados são oriundos do OpenFlights (<https://openflights.org/data>) e datam de 2014:

- **airports.csv:** Contém informações detalhadas sobre aeroportos, incluindo **Airport ID**, **IATA**, **Latitude** e **Longitude**.
- **routes.csv:** Lista as rotas de voo, contendo **Source airport ID** e **Destination airport ID**.

4.2 Fluxo de Funcionamento do `conversor.py`

O `conversor.py` é o script de pré-processamento, essencial para estruturar o grafo:

1. **Leitura de Coordenadas:** O script lê o `airports.csv`, filtrando os aeroportos que possuem um código IATA válido. Armazena um mapa $\{IATA \rightarrow (\text{Latitude}, \text{Longitude})\}$.
2. **Cálculo de Pesos (Arestas):** O script lê o `routes.csv`. Para cada rota $A \rightarrow B$, ele recupera as coordenadas de A e B do mapa e calcula a distância real em km usando a **Fórmula de Haversine**.
3. **Geração de Saídas:** Produz dois arquivos.
 - **airports_processed.csv:** Tabela $IATA \rightarrow (\text{lat}, \text{lon})$. Usada pelo A^* para o cálculo da heurística.
 - **edges.csv:** Lista de arestas direcionadas: *source, destination, weight_km*. Usada para montar o grafo na memória.

A complexidade de tempo desta etapa é $O(|A| + |R|)$, sendo linear e altamente eficiente.

5 Heurística e a Fórmula de Haversine

5.1 A Fórmula de Haversine

A Fórmula de Haversine é usada para calcular a distância de grande círculo entre dois pontos em uma esfera, dadas suas coordenadas geográficas (latitude e longitude). É a fórmula correta para calcular as distâncias aéreas, pois considera a curvatura da Terra, aproximada de uma esfera.

A fórmula é definida como:

$$d = 2R \arcsin \left(\sqrt{\sin^2 \left(\frac{\phi_2 - \phi_1}{2} \right) + \cos(\phi_1) \cos(\phi_2) \sin^2 \left(\frac{\lambda_2 - \lambda_1}{2} \right)} \right)$$

Onde:

- $R = 6371$ km (Raio médio da Terra).
- ϕ_1, λ_1 : Latitude e Longitude do Ponto 1 (em radianos).

- ϕ_2, λ_2 : Latitude e Longitude do Ponto 2 (em radianos).

Uso no Projeto:

- **Peso da Aresta ($g(n)$):** Usada no `conversor.py` para calcular o peso real de cada voo em `edges.csv`.
- **Heurística ($h(n)$):** Usada no `astar.cpp` para calcular a distância em linha reta do nó atual ao destino.

5.2 Justificativa da Heurística

A heurística $h(n)$ utilizada é a **distância geodésica em linha reta (Haversine)** entre o aeroporto n e o destino. Esta é uma solução ideal porque:

- **Admissibilidade:** Uma heurística é **admissível** se nunca superestima o custo real do caminho. A distância em linha reta é, por definição, o caminho mais curto entre dois pontos; portanto, o custo estimado $h(n)$ será sempre menor ou igual ao custo real (o caminho de voos que compõe o caminho mais curto). A admissibilidade garante que o A^* encontrará o caminho **ótimo** (o mais curto).
- **Consistência:** Esta heurística também é **consistente** no contexto geográfico, o que otimiza a busca, pois as distâncias reais de voo raramente são significativamente piores do que a distância em linha reta, permitindo que o A^* explore drasticamente menos nós que o Dijkstra.

6 Fluxo e Análise do `astar.cpp`

6.1 Fluxo de Funcionamento

O programa `astar.cpp` executa a busca do caminho mais curto:

1. **Carga de Dados:** Carrega `airports_processed.csv` e `edges.csv`, montando o grafo e a tabela de coordenadas em memória. Utiliza `std::unordered_map` para acesso rápido.
2. **Consulta:** Solicita ao usuário os códigos IATA de origem (A) e destino (B).
3. **Inicialização do A^* :** O nó inicial é inserido em uma **Fila de Prioridade** (`std::priority_queue` como min-heap), priorizando o menor valor de $f(n)$.
4. **Loop de Busca:** O A^* extrai o nó com o menor $f(n)$ da fila e expande seus vizinhos, calculando o novo custo $g_{\text{tentativo}} = g(n) + \text{peso da aresta}$.
5. **Atualização:** Se $g_{\text{tentativo}}$ for menor do que o custo g registrado para o vizinho, o caminho é atualizado, o predecessor (**parent**) é registrado e o novo $f(n)$ é inserido na fila.
6. **Resultado:** Ao alcançar o nó de destino, o caminho é reconstruído a partir do mapa **parent** e o custo total é exibido.

6.2 Análise da Complexidade e Estruturas

- **Estruturas de Dados:** O uso de `std::unordered_map` para o grafo, custos (g) e predecessores (`parent`) garante um tempo de acesso médio $O(1)$ para cada nó, otimizando a busca.
- **Complexidade de Tempo:** A complexidade do A^* no pior caso é a mesma do Dijkstra, $O((|E| + |V|) \log |V|)$, devido à operação da fila de prioridade.
- **Eficiência Prática:** Dada a **admissibilidade e consistência** da heurística de Haversine, o A^* explora significativamente menos nós do que o Dijkstra, concentrando a busca na direção do destino. Isso resulta em um tempo de execução prático muito próximo de $O(|V_{\text{explorados}}| \log |V|)$, onde $|V_{\text{explorados}}|$ é muito menor que o total de nós $|V|$.

7 Manual de Execução

1. Certifique-se de que os arquivos de entrada (`airports.csv` e `routes.csv`) estão no mesmo diretório.
2. **Execução do Pré-Processamento (Python):**

```
python3 conversor.py
```

Este comando gera os arquivos `airports_processed.csv` e `edges.csv`.

3. **Compilação e Execução do Algoritmo (C++):**

```
g++ astar.cpp -o astar.out && ./astar.out
```

O programa solicitará que você digite os códigos IATA de origem e destino (ex.: GRU LAX).

8 Conclusão

O projeto demonstrou a aplicação bem-sucedida de conceitos de Teoria dos Grafos e Complexidade de Algoritmos para resolver um **problema de otimização no mundo real**. A escolha do Algoritmo A^* foi crucial, pois combinou a garantia de otimalidade do Dijkstra com a **alta eficiência da busca heurística**. A utilização da Fórmula de Haversine para calcular a heurística se provou admissível e consistente, garantindo que o A^* encontrasse o caminho mais curto real, explorando apenas uma fração dos nós do grafo total. O resultado é um sistema rápido e preciso para determinar rotas aéreas ótimas.

9 Referências

Referências

- [1] OpenFlights. *Airline Route and Airport Data*. Disponível em: <https://openflights.org/data>. Acesso em: 1 de Dez. 2025.
- [2] SINE, J. R. The Haversine Formula. In: *Trigonometry and Navigation*.